

Salary Management System

Frontend Approach & Feature Overview — aligned with the assessment requirements

1. Frontend Approach

React + TypeScript + Vite + Tailwind CSS + TanStack Query frontend designed for HR to search, manage and analyze compensation without interacting with APIs directly. The UI is organized into three product areas: **Overview** → **Workforce** → **Configuration**.

The frontend complements the backend architecture by keeping data retrieval server-driven, making 10,000+ employee records manageable, and presenting salary analytics in a clear HR-focused workflow.

2. Problems Solved

HR problem	Frontend solution
Raw APIs are difficult for HR users	Feature pages provide forms, tables, filters and charts instead of direct API interaction
10,000+ rows cannot efficiently load in the browser	Server-side filtering, Apply action, pagination and scoped API requests
Filters can be lost when navigating	Session storage preserves Dashboard, Employees and Salary Records filter scope within the tab
Country and currency filters can duplicate the same scope	Currency-focused filtering while country remains available as an analytics breakdown
Multi-country compensation is hard to compare visually	Dashboard reporting currency plus charts and country/department/level breakdowns
Analytics users need to inspect underlying employees	Drill-down links open Employees or Salary Records with filters carried in the URL
Hiring requires employee and package information	Two-step Add Employee wizard: create employee → configure salary package
Configuration is scattered across the product	Master Data, Salary Components and Exchange Rates are grouped under Configuration
Long tables are difficult on mobile	Responsive card presentation for employee and salary list pages

3. Frontend Features

Dashboard — Compensation Analytics: Date presets/range or snapshot, employee and payment filters, Apply-to-report workflow, Overview/Compensation/Payments/Trends tabs, KPI cards, charts, country/department/level tables, CSV export and drill-down.

Employees: Searchable and filterable employee list by department, job level, status, currency and salary band; sorting, pagination, two-step creation and detail pages for profile, salary history and edits.

Salary Records: Organization-wide period-pay list with date range, payment status and record type filters; summary metrics and employee-context editing.

Master Data: CRUD interfaces for countries, currencies, departments and job levels.

Salary Components: Management of the earnings/deductions catalog used when creating salary packages and records.

Exchange Rates: View live FX pairs used for reporting and cross-currency analytics.

Shared UX: Sidebar application shell, multi-select filters, active-scope chips, loading/empty states and friendly API error handling.

4. Frontend Structure

Path	Responsibility
app/	Application shell, routing, layout and React Query setup
features/	One folder per business area containing pages, local filters and feature UI
shared/api/	HTTP client and typed API helpers
shared/ui/	Reusable buttons, tables, chips, modals and common UI components

Path	Responsibility
shared/lib/	Cross-cutting helpers such as session filter persistence

5. Core UX Pattern

Draft filters → **Apply** → **Fetch**. The UI does not dump the full workforce dataset into the browser. Instead, it requests only the current page and scope from the backend. The selected scope is preserved during navigation, and analytics can drill into the same filtered workforce views.

6. Backend ↔ Frontend Mapping

Backend capability	Frontend surface
Employees API	Employees list and employee detail
Salary profiles / records	Employee creation wizard, salary detail/history and Salary Records
Master data / salary components / FX	Configuration pages
Filtered analytics and breakdowns	Dashboard
Pagination / filters / FX	Shared filter panels and reporting-currency controls

7. Assessment Alignment

The frontend directly supports the assessment's HR Manager persona and the need to manage and understand compensation for approximately 10,000 employees. It provides a usable web interface over the relational backend, avoids browser-side bulk loading, supports search/filter/pagination, exposes analytics and drill-down workflows, and keeps configuration capabilities organized. The design favors clarity and maintainability over unnecessary UI complexity.

8. End-to-End Architecture

HR User → **React UI** → **TanStack Query** → **Typed API Client** → **FastAPI Backend** → **Services** → **Repositories** → **PostgreSQL**

■ FX / Analytics

Note: This document describes the frontend approach supplied for the Salary Management System and uses the current product terminology and feature boundaries.